

ANÁLISIS DE RENDIMIENTO CON SQL PROFILER Y EXTENDED EVENTS

CONOCE, MIDE, ANALIZA Y OPTIMIZA TU BASE DE DATOS

"Así como Yozuru Hanyu perfecciona cada movimiento en el hielo, nosotros analizamos cada evento para lograr el mejor rendimiento en nuestra base de datos."

¿QUÉ ES?



SQL Profiler

Herramienta tradicional de SQL Server que captura eventos en tiempo real para monitorear el comportamiento de consultas, recursos y errores.



Extended Events

Nueva generación de monitoreo más ligera, flexible y con mejor rendimiento que SQL Profiler.

¿POR QUÉ ES IMPORTANTE?



Identifica consultas lentas y cuellos de botella.



Analiza el uso de recursos del servidor.



Detecta bloqueos, esperas y errores.



Permite tomar decisiones para optimizar el rendimiento de la base de datos.

"No se trata de trabajar más, sino de trabajar con precisión."

— GUADALUPE CARBAJAL EMANUEL GADIEL

NUESTRA BASE DE DATOS: GradosTitulos

Descripción

Base de datos que gestiona los procesos académicos relacionados con Grados y Títulos de Pregrado según el Reglamento General de la UPLA.

Tablas principales

- ✓ Estudiantes
- ✓ EscuelaProfesional
- ✓ ModalidadExámenes
- ✓ TipoDocumento
- ✓ GradosTitulos
- ✓ DetalleGradosTitulos
- ✓ Asesores
- ✓ JuradoEvaluador

Esquema Relacional (vista simplificada)



ASÍ OPTIMIZAMOS NUESTRA BASE DE DATOS COMO UN CAZADOR

- ★ Rendimiento óptimo
- ★ Consultas eficientes
- ★ Decisiones informadas
- ★ Base de datos saludable

1. SQL PROFILER

Herramienta clásica para capturar eventos en tiempo real.

¿CÓMO FUNCIONA?

- ✓ Abrir SQL Server Profiler.
- ✓ Conectar al servidor.
- ✓ Seleccionar eventos.
- ✓ HPCCommand, SQL:BatchCompleted.
- ✓ Ejecutar las consultas a examinar en la base de datos.
- ✓ Detener la captura y analizar.



EVENTOS COMUNES EN SQL PROFILER

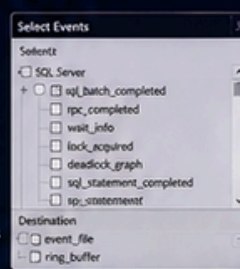
- SQL:BatchCompleted
- Audit Login
- RPC:Completed
- Lock:Deadlock
- HPCCommand
- Errors and Warnings

2. EXTENDED EVENTS

Herramienta moderna, ligera y con mejor rendimiento.

¿CÓMO FUNCIONA?

- ✓ Crear una sesión Extended Events.
- ✓ Seleccionar eventos (sql_batch_completed, wait_info, lock_acquired).
- ✓ Definir el destino (archivo o ring_buffer).
- ✓ Iniciar la sesión.
- ✓ Analizar los datos capturados con SSMS.



EVENTOS RECOMENDADOS EN EXTENDED EVENTS

- sql_batch_completed
- lock_acquired
- rpc_completed
- errors_reported
- wait_info
- deadlock_graph

COMPARATIVO RÁPIDO

SQL PROFILER	VS	EXTENDED EVENTS
Impacto en el servidor	Alto	Bajo
Rendimiento	Menor	Mayor
Flexibilidad	Limitada	Alta
Almacenamiento	Mayor consumo	Eficiente
Recomendación	Sólo para diagnóstico puntual	Ideal para monitoreo continuo

"La diferencia entre nivel y poder está en los detalles.
La diferencia entre datos y visión está en el análisis."

3. EJEMPLO PRÁCTICO CON NUESTRA BASE DE DATOS: GradosTitulos

1 Ejecutamos una consulta que consume recursos.

```
SELECT e.Nombres, ep.Escuela,
       gt.TituloAdmnico,
       gt.FechaGrado
FROM Estudiantes e
JOIN EscuelaProfesional ep
  ON e.IdEscuela = ep.IdEscuela
JOIN GradosTitulos gt
  ON e.IdEstudiante = gt.IdEstudiante
WHERE gt.IdTipoDocumento = '011'
ORDER BY gt.FechaRegistro DESC, 1;
```

2 Capturamos con Profiler o Extended Events.



SQL Profiler

Extended Events

3 Analizamos los resultados.

- Identificamos consultas lentas.
- Medimos tiempos de CPU y duración.
- Detectamos esperas (waits).
- Analizamos bloqueos.
- Revisamos uso de índices.

4 Optimizamos y mejoramos el rendimiento.

- ✓ Crear índices adecuados.
- ✓ Reescribir consultas ineficientes.
- ✓ Actualizar estadísticas.
- ✓ Reducir bloqueos y esperas.
- ✓ Monitorear continuamente.



BENEFICIOS DEL ANÁLISIS DE RENDIMIENTO



Mejor velocidad de consultas



Uso eficiente de recursos



Menos errores y bloqueos



Mejor experiencia de usuario



Decisiones basadas en datos



Rendimiento sostenible en el tiempo



ESTADÍSTICAS E ÍNDICES

(CREACIÓN, FRAGMENTACIÓN, MANTENIMIENTO)

MEJORA EL RENDIMIENTO DE TU BASE DE DATOS

"Como en el hielo, cada decisión cuenta."
"Buenas estadísticas e índices te llevan a un rendimiento perfecto."

¿QUÉ SON?



ESTADÍSTICAS

Son información que SQL Server utiliza para estimar la selectividad y el costo de ejecución de las consultas y elegir el mejor plan de ejecución.



ÍNDICES

Son estructuras que mejoran la velocidad de búsqueda y acceso a los datos, similar al índice de un libro. Ayudan al optimizador de consultas.

¿POR QUÉ SON IMPORTANTES?



Mejoran el rendimiento de las consultas.



Permiten mejores planes de ejecución en SQL Server.



Reducen el I/O y el consumo de recursos.



Aumentan la eficiencia y escalabilidad.

NUESTRA BASE DE DATOS: GradosTitulos

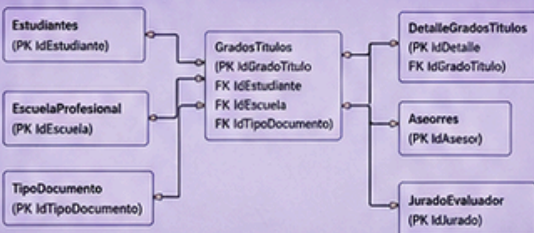


Base de datos que gestiona estudiantes, títulos y relaciones académicas en la Facultad de Ingeniería de la UPLA, según el Reglamento General.

Tablas principales:

- Estudiantes
- EscuelaProfesional
- ModalidadExámenes
- TipoDocumento
- GradosTitulos
- DetalleGradosTitulos
- Asesores
- JuradoEvaluador

Vista relacional simplificada



ÍNDICES Y ESTADÍSTICAS EN ACCIÓN

Una pequeña optimización hoy evita grandes problemas mañana.



1. ESTADÍSTICAS

SQL Server crea estadísticas automáticamente, pero es importante revisarlas y actualizarlas para que el optimizador genere planes eficientes.

TIPOS DE ESTADÍSTICAS

- ✓ Automáticas (creadas por SQL Server)
- ✓ De columna única
- ✓ De múltiples columnas
- ✓ De índices (sobre índices)

¿CÓMO VER ESTADÍSTICAS?

```
EXEC sp_helpstats 'tabla', 'columna';  
-- O bien  
SELECT * FROM sys.stats  
WHERE object_id = OBJECT_ID('tabla');
```

ACTUALIZAR ESTADÍSTICAS

```
-- Actualizar todas las estadísticas de la base de datos  
EXEC sp_updatestats;  
-- O actualizar una tabla específica  
UPDATE STATISTICS tabla_con_nombre;
```

2. ÍNDICES

Los índices aceleran las consultas, pero un mal uso puede afectar la operación de inserción, actualización y eliminar datos.

TIPOS DE ÍNDICES

- ✓ Clustered (agrupa datos físicamente)
- ✓ Nonclustered (índice secundario)
- ✓ Índices únicos (unique)
- ✓ Índices filtrados (filtered)
- ✓ Columnstore (para grandes volúmenes)

CREAR ÍNDICES

```
-- índice no clusterizado en estudiante  
CREATE INDEX IX_Estudiante_Nombre  
ON Estudiantes (Nombre);  
-- índice compuesto en DetalleGradosTitulos  
CREATE INDEX IX_Detalle_GradoTitulo_Estudiante  
ON DetalleGradosTitulos (IdEstudiante, IdGradoTitulo);  
-- índice clusterizado en GradosTitulos  
CREATE CLUSTERED INDEX PK_GradosTitulos  
ON GradosTitulos (IdGradoTitulo);
```

BENEFICIOS DE LOS ÍNDICES



Búsquedas más rápidas



Ordenamiento eficiente



Uniones (JOIN) más rápidas



Menor tiempo de respuesta

3. FRAGMENTACIÓN DE ÍNDICES

La fragmentación ocurre cuando las páginas del índice no están almacenadas de forma contigua, lo que puede degradar el rendimiento.

TIPOS DE FRAGMENTACIÓN

- Baja: 0 - 30%
- Alta: 30% o más

VERIFICAR

```
SELECT index_name, avg_fragmentation_in_percent,  
page_count, fragment_count  
FROM sys.dm_index_physical_stats  
(0, 0, 0, OBJECT_ID('tabla'), NULL, NULL, 'LIMITED');  
-- Valor recomendado: < 30% (ideal < 10%)
```

RANGOS RECOMENDADOS

Fragmentación	Acción recomendada	Método
0% - 10%	Ninguna	-
10% - 30%	Reorganizar	ALTER INDEX ... REORGANIZE
> 30%	Reconstruir	ALTER INDEX ... REBUILD



4. MANTENIMIENTO DE ÍNDICES

El mantenimiento periódico mantiene los índices y estadísticas en óptimas condiciones.

REORGANIZAR ÍNDICE

(Para fragmentación 10% - 30%)
ALTER INDEX IX_Nombre_Indice
ON Tabla
REORGANIZE;



RECONSTRUIR ÍNDICE

(Para fragmentación > 30%)
ALTER INDEX IX_Nombre_Indice
ON Tabla
REBUILD;
-- Con opción ONLINE si es posible



RECONSTRUIR TODA LA TABLA O ÍNDICES

```
EXEC sp_MSforeachtable  
'ALTER INDEX ALL ON ?  
REBUILD';  
-- O usar scripts de Ola Hallengren
```



BUENAS PRÁCTICAS

- ✓ Verificar periódicamente fragmentación.
- ✓ Mantener estadísticas con FULLSCAN en tablas críticas.
- ✓ Evitar índices innecesarios o duplicados.
- ✓ Monitorear fragmentación regularmente.
- ✓ Programar mantenimientos en horarios de bajo uso.



EJEMPLO PRÁCTICO CON GradosTitulos

1. Revisamos índices y fragmentación

2. Detectamos índices con alta fragmentación (>30%)

Índice	Fragmentación %
IX_Estudiante_Nombre	45.2
IX_Detalle_Estudiante	18.6
IX_GradosTitulos_TipoDoc	62.1

3. Reconstruimos el índice

```
ALTER INDEX IX_GradosTitulos_TipoDoc  
ON GradosTitulos  
REBUILD  
WITH (ONLINE = ON);
```

4. Verificamos nuevamente

Índice	Fragmentación %
IX_Estudiante_Nombre	5.1
IX_Detalle_Estudiante	2.8
IX_GradosTitulos_TipoDoc	1.0



BENEFICIOS FINALES



Mejor rendimiento en consultas



Planes de ejecución óptimos



Menos consumo de recursos



Experiencia de usuario mejorada



Mayor estabilidad de la base de datos

DISCIPLINA PARA BASES DE DATOS EXCELENTE

Pequeñas acciones, grandes resultados.





ADMINISTRACIÓN DE TRANSACCIONES Y BLOQUES



INTEGRIDAD, CONCURRENCIA Y RENDIMIENTO PARA TU BASE DE DATOS.

"La precisión en cada movimiento nace del control y la confianza."

— Yozuru Hanyu

¿QUÉ ES?



La administración de transacciones agrupa y asegura operaciones de manera segura y confiable, asegurando la integridad de los datos incluso en fallos del sistema o concurrencia.

¿POR QUÉ ES IMPORTANTE?



Mantiene la integridad y consistencia de los datos.



Permite gestionar múltiples usuarios simultáneos.



Asegura recuperación ante errores o caídas del sistema.



Mejora el rendimiento y eficiencia del sistema.

PROPIEDADES ACID



Atomicidad: La transacción es "todo o nada". Si falla, no se aplica; se revierte.



Consistencia: La transacción mantiene las reglas de integridad de la base de datos.



Aislamiento: Las transacciones concurrentes no se interfieren entre sí.



Durabilidad: Los cambios confirmados permanecen incluso ante fallos.

NUESTRA BASE DE DATOS GradosTítulos (5M)



Estudiantes

• Personas inscritas, datos personales, acceso al sistema.



Titulos

• Titulos obtenidos y fechas.



GradosTítulos

• Trabajos/grados académicos.



EstudianteGrado Titulo

• Intermediario Estudiante - Grado/Titulo.



TipoDocumento

• Documentos de identidad.

Planificamos desde el inicio de la tabla hasta obtener los datos.

NUESTRA BASE DE DATOS: GradosTítulos

Descripción

Base de datos que gestiona los estudiantes, títulos y relaciones de Pregrado de la UPLA, según el Reglamento General.

Tablas principales

- Estudiantes
- EscuelaProfesional
- ModalidadExámenes
- TipoDocumento
- GradosTítulos
- DetalleGradosTítulos
- Asesores
- JuradoEvaluador

Esquema Relacional (vista simplificada)



¿QUÉ SON LOS BLOQUES?



Los bloques (o páginas) son las unidades básicas de almacenamiento en SQL Server. Cada bloque tiene un tamaño fijo (8 KB por defecto) y contiene datos, índices y metadatos.

CICLO DE VIDA DE UNA TRANSACCIÓN



INICIA TRANSACCIÓN

Inicio de la transacción.



OPERACIONES (INSERT, UPDATE, DELETE)

Ejecución de acciones.



COMMIT (ÉXITO)

Cambios guardados de forma permanente.



ROLLBACK (ERROR)

Se deshace todo cambio realizado.

PROBLEMAS DE CONCURRENCIA



Lecturas sucias (Dirty Read)

Lee datos que aún no han sido confirmados.



Lecturas no repetibles (Non-Repeatable Read)

Lee datos que luego cambian.



Lecturas fantasma (Phantom Read)

Aparecen o desaparecen filas nuevas en una consulta.



Actualizaciones perdidas (Lost Update)

Dos transacciones sobrescriben el mismo dato sin coordinarse.

NIVELES DE AISLAMIENTO EN SQL SERVER

Nivel de Aislamiento	Lectura Sucia (Dirty Read)	Lectura No Repetible	Lectura Fantasma	Uso Recomendado
READ UNCOMMITTED	✗	✗	✗	Solo para diagnósticos o entornos sin datos críticos.
READ COMMITTED	✓	✗	✗	Aplicaciones generales con datos críticos.
REPEATABLE READ	✓	✓	✗	fecluras.
SERIALIZABLE	✓	✓	✓	Máxima integridad, menor concurrencia.

TIPOS DE BLOQUES (LOCKS)



Tipo de Lock	Descripción	Ejemplo de Uso
Shared (S)	Permite lectura, bloquea escritura.	SELECT
Exclusive (X)	Bloquea lectura y escritura.	INSERT, UPDATE, DELETE
Update (U)	Permite leer, pero prepara bloqueo exclusivo.	Uso en operaciones de actualización
Intent (IS, IX, IU...)	Indican intención de adquirir otros locks.	Operaciones en múltiples niveles de jerarquía.

EJEMPLO PRÁCTICO CON GradosTítulos

1 Inicio de transacción

Iniciamos la transacción para registrar un nuevo título.

BEGIN TRANSACTION;
-- Operaciones
INSERT INTO GradosTítulos...



2 Operaciones (Insert/Update)

Insertamos o actualizamos los datos relacionados.

INSERT INTO GradosTítulos (IdEstudiante, IdGradoTitulo, Fecha, Estado)
VALUES (...);



3 Confirmar (Éxito)

Confirmamos si todo se ejecutó correctamente.

COMMIT TRANSACTION;
-- Cambios guardados exitosamente.



4 Error (Rollback)

Si ocurre un error, deshacemos todos los cambios.

ROLLBACK TRANSACTION;
-- Cambios revertidos y no guardados.



5 Bloques afectados



■ Bloques usados
■ Bloques libres

"SQL Server gestiona los bloques y la transacción para garantizar que cada operación sea segura, consistente y confiable."



EN RESUMEN

Controlar transacciones y bloques es clave para garantizar integridad, rendimiento y confiabilidad en tu base de datos.



BENEFICIOS DE UNA BUENA ADMINISTRACIÓN



Mejor integridad de los datos



Mayor rendimiento del sistema



Menos errores y fallos críticos



Usuarios más satisfechos



Menos errores de datos



Mayor eficiencia en consultas



Datos confiables siempre



Escalabilidad mejorada



Experiencia de usuario optimizada



OPTIMIZACIÓN DE CONSULTAS T-SQL

ESCRIBE MENOS, OBTÉN MÁS: CONSULTAS RÁPIDAS Y EFICIENTES

Caso práctico con la base de datos: GradosTítulos



¿QUÉ ES LA OPTIMIZACIÓN DE CONSULTAS?

Consiste en escribir consultas T-SQL eficientes que recuperan menos recursos, responden más rápido y escalan mejor.



¿POR QUÉ OPTIMIZAR?

- Reduce tiempos de respuesta.
- Disminuye uso de CPU y memoria.
- Mejora la experiencia del usuario.
- Permite que la base de datos soporte más carga.



REGLA DE ORO

Devuelve solo lo necesario, lo antes posible y usa los índices a tu favor.

1 DIAGNÓSTICO: ¿TU CONSULTA ESTÁ SOBRECARGANDO EL SERVIDOR?

Síntomas de consultas no optimizadas

- ⌚ Tiempos de respuesta altos
- ⏸ Bloqueos y esperas
- 🔥 Alto uso de CPU
- 📖 Lecturas lógicas elevadas
- 💰 Planes de ejecución costosos

Herramientas para detectar problemas

- SET STATISTICS IO, TIME ON
- SQL Server Profiler
- Extended Events
- Execution Plan (Ctrl + M)
- DMV (ej. sys.dm_exec_query_stats)

2 PRINCIPIOS CLAVE PARA OPTIMIZAR CONSULTAS



Filtra primero, selecciona después.



Devuelve solo lo que necesitas.



Usa índices de manera adecuada.



Usa eficientemente las uniones.



Mantén consultas actualizadas.

Menos datos leídos • Menos trabajo para el servidor • Más rendimiento

"No hacer por el mejor, sino hacer lo mejor con lo que se tiene."

- Yozuru Hanyu

3 MEJORES PRÁCTICAS Y EJEMPLOS

PRÁCTICA	EVITA ❌	MEJOR USA ✅	EXPLICACIÓN
★ 1. Evitar SELECT *	SELECT * FROM Academia.Persona WHERE Estado = 'A'	SELECT PK_Persona, Nombres, Apellidos, DNI FROM Academia.Persona WHERE Estado = 'A'	Tráe solo las columnas necesarias. Menos datos = más velocidad.
🔍 2. Usar filtros selectivos	SELECT * FROM Transito.Transito WHERE CONVERT(date, FechaRegistro) = '2024-01-01'	SELECT * FROM Transito.Transito WHERE FechaRegistro >= '2024-01-01' AND FechaRegistro < '2024-01-02'	Permite usar índices en la columna FechaRegistro.
fx 3. Evitar funciones en columnas	SELECT * FROM Academia.Persona WHERE UPPER(Nombres) = 'JUAN'	SELECT * FROM Academia.Persona WHERE Nombres LIKE 'Juan%' COLLATE SQL_Latin1_General_CP1_CI_AS	Las funciones impiden el uso de índices.
🔗 4. Usar JOIN en vez de subconsultas	SELECT * FROM Persona P, Matricula M WHERE P.IdPersona = M.IdEstudiante	SELECT * FROM Persona P INNER JOIN Matricula M ON P.IdPersona = M.IdEstudiante	Más legible y permite al optimizador elegir mejores planes.
🔄 5. Usar EXISTS en lugar de IN	SELECT * FROM Transito.Tramite WHERE IdPersona IN (SELECT IdPersona FROM Transito.D_T_Trabaja)	SELECT * FROM Transito.Tramite T WHERE EXISTS (SELECT 1 FROM Transito.D_T_Trabaja D WHERE D.IdPersona = T.IdPersona)	EXISTS puede ser más eficiente en grandes volúmenes.
⌚ 6. Evitar OR innecesarios	SELECT * FROM Persona WHERE Estado = 'A' OR 'I' OR 'P'	SELECT * FROM Persona WHERE Estado IN ('A', 'I', 'P')	IN es más eficiente que múltiples OR.

TIPS RÁPIDOS

- ✅ Usa índices en columnas usadas en WHERE, JOIN, ORDER BY y GROUP BY.
- ✅ Actualiza estadísticas regularmente.
- ✅ Evita subconsultas anidadas innecesarias.
- ✅ Usa TOP con ORDER BY o cuando solo necesitas las primeras registros.
- ✅ Revisa el plan de ejecución antes de optimizar.

4 CASO PRÁCTICO: CONSULTA MAL VS BIEN OPTIMIZADA

CONSULTA NO OPTIMIZADA (EVITAR)

```
SELECT *
FROM Transito.Tramite T, Academia.Persona P,
D_T_Trabaja DT, A_T_TrabajaArea A
WHERE CONVERT(date, T.FechaRegistro) = '2024-01-01'
AND T.IdPersona = P.IdPersona
AND DT.IdPersona = P.IdPersona
AND A.IdTrabaj = DT.IdTrabaja;
```

VS

CONSULTA OPTIMIZADA (MEJOR)

```
SELECT P.PK_Persona, P.Nombres, P.Apellidos, T.Istado
FROM Transito.Tramite T
INNER JOIN Academia.Persona P ON P.IdPersona = T.IdPersona
INNER JOIN D_T_Trabaja DT ON DT.IdPersona = P.IdPersona
INNER JOIN A_T_TrabajaArea A ON A.IdTrabaja = DT.IdTrabaja
WHERE T.FechaRegistro >= '2024-01-01'
AND T.FechaRegistro < '2024-01-02';
```

¿QUÉ MEJORÓ?



Mejora promedio: 70% en lecturas, y 94% menos tiempo de ejecución.

5 ÍNDICES QUE ACELERAN TUS CONSULTAS

TABLA	ÍNDICE RECOMENDADO	¿PARA QUÉ SIRVE?
Persona	IX_Persona_DNI	Búsquedas por DNI
Persona	IX_Persona_Estado	Filtrar por estado
Transito.Tramite	IX_Tramite_FechaRegistro	Filtrar por rango de fechas
Transito.Tramite	IX_Tramite_IdPersona	Uniones con otras tablas
Academia.Persona	IX_Persona_Nombres	Búsqueda por nombres
D_T_Trabaja	IX_DT_IdPersona	Uniones eficientes
A_T_TrabajaArea	IX_Area_IdTrabaja	Consultas por área/trabaja



TIP: No corras detrás de consultas. Crea índices donde el servidor corre más rápido.



6 CHECKLIST DE OPTIMIZACIÓN

- ✅ ¿Filtros aplicados en columnas indexadas?
- ✅ ¿Devuelvo solo las columnas necesarias?
- ✅ ¿Evito funciones sobre columnas?
- ✅ ¿Uso JOIN en lugar de subconsultas?
- ✅ ¿Los índices soportan mis consultas?
- ✅ ¿Actualicé estadísticas regularmente?
- ✅ ¿Consulté el plan de ejecución?
- ✅ ¿Eliminé OR's e redundancias innecesarias?
- ✅ ¿Probé con datos reales y representativos?



La precisión en la lógica de la consulta es la ejecución en el servidor.

7 ESTUDIANTE

Estudiante:
Guadalupe Carbajal
Emanuel Gadiel



MENOS DATOS LEÍDOS
MEJOR RENDIMIENTO



CONSULTAS EFICIENTES
MAYOR VELOCIDAD



ÍNDICES INTELIGENTES
MENOS RECURSOS



PLANES ÓPTIMOS
MENOS COSTO



BUENAS PRÁCTICAS
RESULTADOS EXCELENTE



CONTROL DE RECURSOS CON RESOURCE GOVERNOR

Administra, limita y prioriza los recursos del servidor SQL Server

Caso práctico con la base de datos: GradosTítulos



1 ¿QUÉ ES RESOURCE GOVERNOR?



Es una característica de SQL Server que permite administrar y controlar el uso de recursos del sistema (CPU, memoria, E/S y concurrencia) por diferentes grupos de trabajo o cargas.

BENEFICIOS

- ✓ Evita que una carga pesada afecte a otras.
- ✓ Mejora procesos críticos.
- ✓ Mejora la estabilidad del servidor.
- ✓ Asegura un rendimiento justo y predecible.

2 ¿POR QUÉ ES IMPORTANTE?



Ambientes compartidos (producción, reportes, ETL, usuario).



Consultas mal escritas que consumen todos los recursos.



Procesos en segundo plano que compiten con los demás.



Mantener la base de datos GradosTítulos siempre disponible y rápida.

"El verdadero control no es limitar, es dar a cada proceso lo que necesita para alcanzar su mejor nivel."

— Yozuru Hanyu

NUESTRA BASE DE DATOS: GradosTítulos

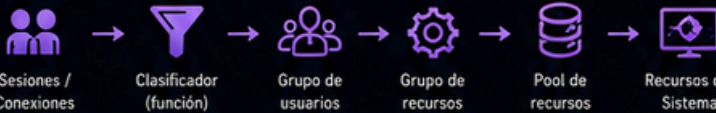
Esquema principal

Académico	Tablas maestros: Persona, Docente, Matricula, ProgramaAcadémico, etc.
Trámite	Procesos de trámites: Títulos, Expedientes, Trámites, etc.
Investigación	Trabajos de investigación, tesis y tipos.
Evaluación	Jurados, evaluaciones, sustentaciones.
Documentos	Resoluciones, diplomas, anexos.

Cargas comunes en el sistema

Usuarios OLTP (operaciones diarias): consultas, carga y transacciones.
Reportes y consultas analíticas lecturas pesadas.
Procesos administrativos (trámites, validaciones).
Mantenimiento y ETL (backups, index rebuilds).
Consultas ad-hoc o desarrollo consultas no controladas.

3 ARQUITECTURA DE RESOURCE GOVERNOR



El clasificador asigna cada sesión a un grupo de usuarios, este mapea a un grupo de recursos, el cual obtiene los recursos de un pool definido.

4 FUNCIÓN CLASIFICADORA (EJEMPLO)

Permite decidir a qué grupo de usuario pertenece cada sesión según el login, aplicación o contexto.

```
CREATE FUNCTION dbo.fn_ClassifierSession()
RETURNS sysname
WITH SCHEMABINDING
BEGIN
    DECLARE @grupo sysname;
    IF APP_NAME() LIKE 'Reportes%'
        SET @grupo = 'GRUPO_REPORTES';
    ELSE IF APP_NAME() LIKE 'Mantenimiento%'
        SET @grupo = 'GRUPO_MANTENIMIENTO';
    ELSE IF ORIGINAL_LOGIN() IN ('academico', 'app_web')
        SET @grupo = 'GRUPO_OLTP';
    ELSE
        SET @grupo = 'GRUPO_ADHOC';
    RETURN @grupo;
END
```

Criterios usados:

- ✓ APP_NAME(): Nombre de la aplicación.
- ✓ ORIGINAL_LOGIN(): Usuario que se conecta.
- ✓ Retorna el nombre del grupo de recursos.



5 GRUPOS DE RECURSOS Y POOLS (EJEMPLO)

Definimos pools (límites máximos) y grupos (qué tanto puede usar dentro del pool).

POOL DE RECURSOS	CPU MAX	MEMORIA MAX	DESCRIPCIÓN
pool_erp	40%	60%	Para operaciones diarias (trámites, consultas OLTP).
pool_reportes	30%	25%	Para reportes y consultas analíticas.
pool_mantenimiento	10%	8%	Para tareas de mantenimiento y ETL.
pool_administrativo	10%	7%	Para administrativos y ad-hoc.

GRUPO DE USUARIOS → GRUPO DE RECURSOS

GRUPO DE USUARIOS	GRUPO DE RECURSOS	POOL ASIGNADO	CPU MAX	MEMORIA MAX	PRIORIDAD
GRUPO_OLTP	grp_oltp	pool_erp	100%	100%	ALTA
GRUPO_REPORTES	grp_reportes	pool_reportes	100%	100%	MEDIA
GRUPO_MANTENIMIENTO	grp_mantenimiento	pool_mantenimiento	100%	100%	MEDIA
GRUPO_ADMINISTRATIVO	grp_admin	pool_administrativo	100%	100%	BAJA

Menor prioridad = primero en recibir menos recursos si el servidor se congestiona.

6 COMANDOS CLAVE (T-SQL)

CREATE FUNCTION (clasificador)

```
CREATE FUNCTION dbo.fn_ClassifierSession()
RETURNS sysname
WITH SCHEMABINDING
AS
... lógica de clasificación
RETURN @grupo;
```

CREATE WORKLOAD GROUP (ejemplo)

```
CREATE WORKLOAD GROUP grp_oltp
WITH (REQUEST_MAX_CPU_TIME_SEC = 0,
MIN_CPU_PERCENT = 0,
IMPORTANCE = HIGH);
```

CREATE RESOURCE POOL

```
CREATE RESOURCE POOL pool_oltp
WITH (MAX_CPU_PERCENT = 40,
MAX_MEMORY_PERCENT = 60);
GO
```

CREATE USER GROUP

```
CREATE USER GROUP GRUPO_OLTP
WITH (USER_LOGIN = 'app_web',
USER_LOGIN = 'academico');
```

CREATE WORKLOAD GROUP

```
CREATE WORKLOAD GROUP grp_oltp
WITH (IMPORTANCE = HIGH,
MAX_CPU_PERCENT = 100,
MAX_MEMORY_PERCENT = 100);
GO
```

CREATE RESOURCE GOVERNOR

```
CREATE RESOURCE GOVERNOR
WITH (RESOURCE_POOL = pool_oltp)
ALTER RESOURCE GOVERNOR RECONFIGURE;
GO
```

IMPORTANTE: Después de crear o modificar, ejecuta ALTER RESOURCE GOVERNOR RECONFIGURE.

7 MONITOREO Y RESULTADOS

Consulta las vistas dinámicas para verificar el consumo por grupo de recursos.

VISTAS ÚTILES

- sys.dm_resource_governor_resource_pools
- sys.dm_resource_governor_workload_groups
- sys.dm_resource_governor_resource_pools por grupo de trabajo.
- sys.dm_exec_sessions Sesiones activas y su grupo asignado.
- sys.resource_governor_configuration Configuración actual.

IMPACTO EN GradosTítulos

- ✓ Las operaciones transaccionales (OLTP) siempre responden.
- ✓ Los reportes ya no afectan la experiencia del usuario.
- ✓ Las tareas de mantenimiento no saturan el servidor.
- ✓ El sistema es más estable y productivo.

VS

SIN RESOURCE GOVERNOR

- ✗ Una consulta pesada puede saturar CPU.
- ✗ Los reportes empeoran la velocidad del sistema.
- ✗ Los procesos críticos se ven afectados.
- ✗ Riesgo de bloqueos y tiempos muertos.



8 MEJORES PRÁCTICAS



Define bien las cargas principales.



Empezar con límites conservadores y ajusta.



Monitorea constantemente el sistema.



Actualiza estadísticas e índices para mejores resultados.



Combina con buenas políticas de seguridad y respaldo.

"La optimización no es magia, es inteligencia aplicada para que cada recurso brille en su lugar."

— Yozuru Hanyu

ESTUDIANTE

Estudiante:
Guadalupe Carbajal
Emanuel Gadiel